Lecture 2: How Python Runs Programs

Dr. Aafaq · B.Tech. CSE, 5th Semester · Compact notes (Lutz, Ch. 2–3)

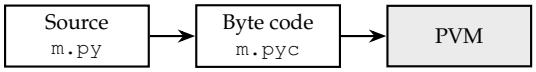
1 The Interpreter

An **interpreter** is a program that runs other programs: a software layer between your code and the hardware. Installing Python gives an interpreter plus a standard library. The standard interpreter, **CPython**, is written in C. Check it with `python -version` (Windows also `py`, Linux/macOS `python3`).

2 Program Execution

A program is a **text file of statements** (`.py`), run top to bottom:

```
print('hello world')
print(2 ** 100)    # ints never overflow
```



See it with the disassembler:

```
import dis
def area(r): return 3.14159 * r * r
dis.dis(area)   # LOAD_CONST, LOAD_FAST,
                # BINARY_OP (*), ... RETURN_VALUE
```

Behind the scenes there are two hidden steps:

1. **Compile** each statement to **byte code**: low-level, platform-independent, not machine code.
2. The **PVM** (Python Virtual Machine), a loop inside the interpreter, **executes** the byte code one instruction at a time.

- Byte code of **imported** files is cached in `__pycache__/` (e.g. `helper.cpython-313.pyc`) and reused if the source is unchanged. The main script is compiled in memory every run.
- **Speed:** no build step; slower than C (byte code is not machine code), so heavy work uses C-based libraries (NumPy).
- **Development:** the compiler is always present, so code can even be built and run at runtime (`eval`, `exec`).

Key idea. Python is both compiled (to byte code) and interpreted (by the PVM), automatically.

3 Execution Model Variations

Implementation	Built on	Use
CPython	C	Standard (python.org, Anaconda)
PyPy	RPython	Faster via a JIT compiler
Jython / IronPython	Java / .NET	Scripting Java / .NET apps
MicroPython	C (tiny)	Microcontrollers: ESP32, Raspberry Pi Pico
Pyodide / PyScript	WebAssembly	Python in the browser

Speed-ups: PyPy (JIT), Cython (compile to C), Numba (JIT for numeric code); CPython 3.13+ has an experimental JIT and a free-threaded build. **Frozen binaries** (PyInstaller) bundle byte code + PVM + libraries into one executable that runs without Python installed. Bigger, not faster.

4 The Interactive Prompt

```
$ python
>>> 2 ** 8          # echoed, no print
256
>>> for i in range(2):
...     print(i)
...                 # blank line ends it
0
1
>>> exit()   # or Ctrl-Z Enter / Ctrl-D
```

Use it to **experiment** and **test**; errors just print a message. Nothing is saved.

Rules:

1. Type Python only (`python f.py` is a shell command).
2. No indentation at top level.
3. End compound statements with a blank line.
4. One statement at a time; `print` needed only in files.

5 Scripts and Command Lines

```
# script1.py
import sys
print(sys.platform)
print(2 ** 100)
x = 'Spam!'
print(x * 8)
```

```
$ python script1.py
$ python script1.py > saveit.txt
```

Gotchas: Windows may hide extensions (`script1.py.txt`); give the full name; `cd` to the folder first.
Unix executable: first line `#!/usr/bin/env python3`, then `chmod +x f.py` and run `./f.py`.
Icon click: the window closes at the end; add `input('Press Enter')` as the last line. Errors vanish too, so not for development.

Dr. Aafaq

6 Imports, Reloads and Namespaces

```
>>> import script1     # runs it (no .py)
>>> import script1     # nothing: once only
>>> from importlib import reload
>>> reload(script1)    # runs edited file
>>> import threenames  # a, b, c = ...
>>> threenames.b       # attribute
'parrot'
>>> from threenames import a, c
>>> dir(threenames)    # [..., 'a','b','c']
```

- Every `.py` file is a **module**; `import` finds, compiles and runs it **once per session**.
- After editing, use `reload`.
- A module is a **namespace** (package of names), so `x` in two files never clashes.
- `exec(open('f.py').read())` runs a file in *your* namespace each time; it can silently overwrite variables. Prefer `import`.

7 IDEs, Launch Options and Debugging

IDLE: F5 runs the file, Alt-P/Alt-N recall commands, Tab completes. **Others:** VS Code (lab default), Jupyter, Spyder, PyCharm, Colab. Python is also *embedded* in tools such as Blender, FreeCAD and QGIS.

Option	Use when	Option	Use when
`>>>` prompt	testing one idea	icon click	programs for non-programmers
`python f.py`	running scripts, saving output	`import/reload`	reusing code across files
`#!` + `chmod +x`	everyday Linux tools	IDE	writing and debugging

Debugging: (1) read the traceback bottom-up: error, file, line; (2) add `print` calls; (3) IDE debugger; (4) `breakpoint()` / `pdb`. E.g. `ZeroDivisionError` at `average = total / count` means `count` is 0 (empty list).

8 Exercises

1. Define interpreter, source code, byte code and PVM. Draw the execution diagram.
2. Why is byte code saved for imported files but not for the main script?
3. Name four Python implementations and a use for each. Does a frozen binary run faster?
4. Give three rules for the interactive prompt.
5. Why reload a module? Write the statements. What is a namespace?
6. (Lab) Save `print('Hello module world!')` as `module1.py`; run it every way you can. Then type `L = [1, 2]; L.append(L); L` and explain the output.

Answers. 2: it speeds later imports; the script is compiled in memory each run. 3: not faster, same PVM and byte code. 5: imports run once per session; `from importlib import reload; reload(m)`; a namespace is a package of names, one per module. 6: `[1, 2, [...]]`, Python detects the list containing itself.

Read next: Lutz Ch. 4, Introducing Python Object Types. Demos: `lecture02_demo.py`, `code/`.

Dr. Rafat